Coordinate Converter

User guide: ECEF XYZ and CCSDS OEM to WGS84 latitude, longitude, altitude

1. What it does

The script converts positions given as X, Y, Z (distances from the center of the Earth) into latitude, longitude, and altitude on the WGS84 ellipsoid, the same Earth model GPS uses. It reads either a CSV file or a CCSDS OEM orbit file and writes a CSV. It works in chunks, so files with millions of points will not run out of memory.

- **CSV input:** X, Y, Z in meters, already Earth-fixed (ECEF).
- **OEM input:** satellite ephemeris in km, in an Earth-fixed or space-fixed frame. Space-fixed positions are rotated to Earth-fixed at each timestamp first.

2. Installation

You need Python 3. Install the required packages:

```
pip install pyproj numpy pandas
```

Only if you will convert OEM files in a space-fixed frame (EME2000, J2000, GCRF, ICRF, TEME):

```
pip install astropy
```

Tip: the file name has a space in it, so wrap it in quotes on the command line: `"Coordinate Converter.py"`. Or rename it to `xyz_to_lla.py` to make life easier.

3. Basic command

```
python "Coordinate Converter.py" INPUT OUTPUT [options]
```

INPUT is your CSV or OEM file. OUTPUT is the CSV to create. If OUTPUT already exists it is overwritten.

4. Converting a CSV file

Your CSV needs one column each for X, Y, and Z, in **meters**. By default the script looks for columns named `x`, `y`, `z` (case sensitive).

Example input (points.csv):

```
id,x,y,z
1,6378137.0,0.0,0.0
2,0.0,6378137.0,0.0
3,-200000.5,-5200000.0,3600000.0
```

Run it:

```
python "Coordinate Converter.py" points.csv points_lla.csv
```

Output: every original column, plus `lat`, `lon`, `alt`. Point 1 above comes out at lat 0, lon 0, alt 0 (on the equator at the prime meridian).

Different column names

If your columns are named something else, such as X_m, Y_m, Z_m:

```
python "Coordinate Converter.py" points.csv out.csv --x X_m --y Y_m --z Z_m
```

5. Converting an OEM file

Files ending in `.oem` are detected automatically:

```
python "Coordinate Converter.py" orbit.oem orbit_lla.csv
```

If the file has another extension (such as .txt), force OEM mode:

```
python "Coordinate Converter.py" orbit.txt orbit_lla.csv --format oem
```

Minimal example OEM file

```
CCSDS_OEM_VERS = 2.0
CREATION_DATE  = 2026-10-19T00:00:00
ORIGINATOR     = MY_TEAM

META_START
OBJECT_NAME    = SAT-1
OBJECT_ID      = 2026-001A
CENTER_NAME    = EARTH
REF_FRAME      = EME2000
TIME_SYSTEM    = UTC
START_TIME     = 2026-10-19T12:00:00.000
STOP_TIME      = 2026-10-19T12:02:00.000
META_STOP

2026-10-19T12:00:00.000  6778.137  0.000     0.000  0.0 7.668 0.0
2026-10-19T12:01:00.000  6758.400  460.000   0.000  -0.5 7.650 0.0
2026-10-19T12:02:00.000  6699.500  918.000   0.000  -1.0 7.600 0.0
```

Each data line is an epoch followed by X Y Z in **km**. Velocity columns after that are ignored. A file can hold several META/data segments, for example several satellites or frames.

OEM output columns

Column	Meaning
object	OBJECT_NAME (or OBJECT_ID if no name)
epoch	Timestamp exactly as written in the file
time_system	Time system from the META block
x_ecef_m, y_ecef_m, z_ecef_m	Earth-fixed position in meters (after any rotation)
lat, lon	Geodetic latitude and longitude in degrees
alt	Height above the WGS84 ellipsoid in meters

6. What OEM files are supported

Item	Supported values
CENTER_NAME	EARTH only
REF_FRAME (Earth-fixed)	ITRF, ITRF2014, ITRF2020, any ITRF*, EFG, ECEF. Used as is, no astropy needed.
REF_FRAME (space-fixed)	EME2000, J2000, GCRF, ICRF, TEME. Rotated to Earth-fixed (ITRS) with astropy.
TIME_SYSTEM	UTC, TAI, TT, TDB, TCG, TCB, UT1, GPS
Epoch format	2026-10-19T12:00:00.000 or day-of-year 2026-292T12:00:00.000
Skipped	COMMENT lines, COVARIANCE_START ... COVARIANCE_STOP blocks
Not supported	XML OEM files, centers other than Earth

Memory hook: "Fixed spins, Space sits." Earth-fixed frames spin with the planet. Space-fixed frames sit still while the planet turns under them, so the script has to undo that spin for every timestamp.

7. All options

Option	Default	What it does
--format auto|csv|oem	auto	auto treats .oem files as OEM, everything else as CSV
--x NAME	x	X column name (CSV only)
--y NAME	y	Y column name (CSV only)
--z NAME	z	Z column name (CSV only)
--chunksize N	1,000,000 CSV 200,000 OEM	Rows processed per batch. Lower it if you run low on RAM.
--fast	off	Space-fixed OEM only. About 8x faster, under 0.05 mm difference.

8. Speed: when to use --fast

For space-fixed frames, the full rotation is computed at every line by default, which takes about 75 seconds per 500,000 lines. With `--fast`, the slowly changing wobble of Earth's axis (precession-nutation) is computed every 600 seconds and interpolated. Earth rotation and polar motion are still exact at every line.

```
python "Coordinate Converter.py" big_orbit.oem out.csv --fast
```

- Use it for large files. The difference is under 0.05 mm, far below any real orbit accuracy.
- It has no effect on CSV files or Earth-fixed OEM files (no rotation is needed).
- TEME results are exact with or without it.

9. Using it in your own Python code

Rename the file to `xyz_to_lla.py`, put it next to your script, and:

```
from xyz_to_lla import xyz_to_lla

lat, lon, alt = xyz_to_lla(6378137.0, 0.0, 0.0)
print(lat, lon, alt)   # 0.0 0.0 0.0

# Works on lists and numpy arrays too
lat, lon, alt = xyz_to_lla(x_array, y_array, z_array)
```

Inputs are ECEF meters. The return order is **lat, lon, alt**.

10. Troubleshooting

Message or problem	Fix
KeyError: 'x'	Your CSV columns have other names. Use --x, --y, --z.
needs astropy	Run: pip install astropy
looks like an XML OEM file	Only the text (KVN) format is supported. Convert to KVN first.
CENTER_NAME = MOON (or other)	Only Earth-centered orbits are supported.
REF_FRAME ... is not supported	Check the frame name against section 6.
epoch ... is not in a supported format	Use YYYY-MM-DDThh:mm:ss or YYYY-DDDThh:mm:ss.
data line found before any META_START	Every data line needs a META block above it.
Warning about IERS tables	No internet. astropy uses its bundled Earth orientation data, which is fine for most work. Run once online for the latest tables.
Altitudes look about 1000x off	CSV input must be meters. OEM input must be km.
Ran out of memory	Lower --chunksize, for example --chunksize 100000.

11. Quick reference

```
# CSV with default column names
python "Coordinate Converter.py" in.csv out.csv

# CSV with custom columns
python "Coordinate Converter.py" in.csv out.csv --x X --y Y --z Z

# OEM file (auto-detected by .oem)
python "Coordinate Converter.py" orbit.oem out.csv

# OEM with another extension, fast mode
python "Coordinate Converter.py" orbit.txt out.csv --format oem --fast

# Show built-in help
python "Coordinate Converter.py" -h
```